

Final Project - Software Rendering and Shadow Mapping

FINAL PROJECT

Interactive 3D Scene Editor with Real-Time Shadow Mapping

Computer Graphics Course

C++ | VS Code / CLion | MiniFB | MicroUI | GLM | CMake

Student 1	Natalie Bittar ID: 212586044
Student 2	Adan Awad ID:324302678
Course	Computer Graphics
Project	Interactive 3D Scene Editor with Real-Time Shadow Mapping

Submission Date 10/10/2026

Report Structure

1. Project Overview and Objectives
2. Relationship to Course Material
3. Rendering Pipeline and System Architecture
4. Dynamic Scene Editor
5. Real-Time Shadow Mapping
6. Shadow Quality and Debug Visualization
7. Texture Mapping and Perspective-Correct Interpolation
8. Performance Measurements
9. Interaction, Testing, and Final Results
10. Known Limitation and Future Work
11. Conclusion

1. Project Overview and Objectives

The final project transforms the NanoRender software rasterizer into an interactive 3D scene editor. Instead of rendering one fixed model, the application manages a complete scene containing multiple editable objects, a camera, materials, a point light, textures, a ground plane, and a real-time shadow system. The entire result is rendered in software to a framebuffer rather than delegated to a high-level 3D engine.

Main objectives

- Preserve and integrate the rendering pipeline implemented throughout the homework assignments.
- Allow the user to add, select, duplicate, remove, transform, and shade multiple 3D objects.
- Implement real-time shadow mapping using a separate depth pass from the light's point of view.
- Provide multiple shadow-quality modes for comparing visual quality and rendering cost.
- Expose internal rendering data through a shadow-map depth visualization.
- Improve texture and shading accuracy with perspective-correct interpolation.
- Measure FPS, total frame time, and shadow-pass time for performance analysis.

2. Relationship to Course Material

The project is a direct continuation of the topics implemented in the course assignments. It reuses low-level framebuffer rendering and immediate-mode GUI concepts, OBJ mesh loading, model transformations, virtual cameras, perspective projection, triangle rasterization, barycentric coordinates, Z-buffer visibility, back-face culling, material lighting, Flat/Gouraud/Phong shading, and texture mapping.

Course Component	Use in Final Project
OBJ loading and transformations	Multiple scene objects are loaded, normalized, translated, rotated, and scaled.
Camera and projection	The scene is rendered through a perspective camera and viewport transformation.
Barycentric rasterization	Triangles are filled per pixel and attributes are interpolated across each face.
Z-buffer	Nearest visible fragments are selected in the camera pass and in the shadow-depth pass.
Phong reflection model	Ambient, diffuse, and specular lighting are evaluated for shaded pixels.
Texture mapping	UV coordinates sample a BMP texture after perspective correction.

3. Rendering Pipeline and System Architecture

Each frame is processed through two related rendering pipelines. The first optional pass renders depth from the light camera into a shadow map. The second pass renders the visible scene from the user's camera, applies lighting and texture calculations, samples the shadow map, performs the Z-buffer test, and writes the final RGB value into the MiniFB framebuffer.

Main camera pipeline

```
OBJ mesh
-> Model Transformation
-> View Transformation
-> Perspective Projection
-> Perspective Divide
-> Viewport Mapping
-> Triangle Rasterization
-> Barycentric / Perspective-Correct Interpolation
-> Z-Buffer Test
-> Lighting + Texture + Shadow Sampling
-> Framebuffer
```

Per-frame rendering order

1. Read keyboard and mouse input.
2. Update the active object and camera.
3. Build camera view and projection matrices.
4. Render the shadow depth pass when shadows or depth-debug mode are enabled.
5. Clear the color framebuffer and Z-buffer.
6. Render the ground plane.
7. Render all editable scene objects.
8. Optionally replace the viewport with the shadow-map depth visualization.
9. Build and draw the MicroUI interface.
10. Present the framebuffer and update timing statistics.

4. Dynamic Scene Editor

The scene editor converts the renderer from a single-model demonstration into a small interactive graphics application. The model catalog contains cube, pyramid, and octahedron primitives. Each instance keeps its own transformation and material state, while the GUI exposes operations that modify the active object.

- Add cube, pyramid, or octahedron instances.
- Select the active scene object.
- Duplicate the active object.
- Remove objects while preserving at least one scene object.
- Apply automatic grid layout.
- Edit translation, rotation, and uniform scale.
- Edit material properties and shininess.
- Switch between Flat, Gouraud, and Phong shading.
- Toggle textures, wireframe rendering, back-face culling, shadows, and debug vectors.

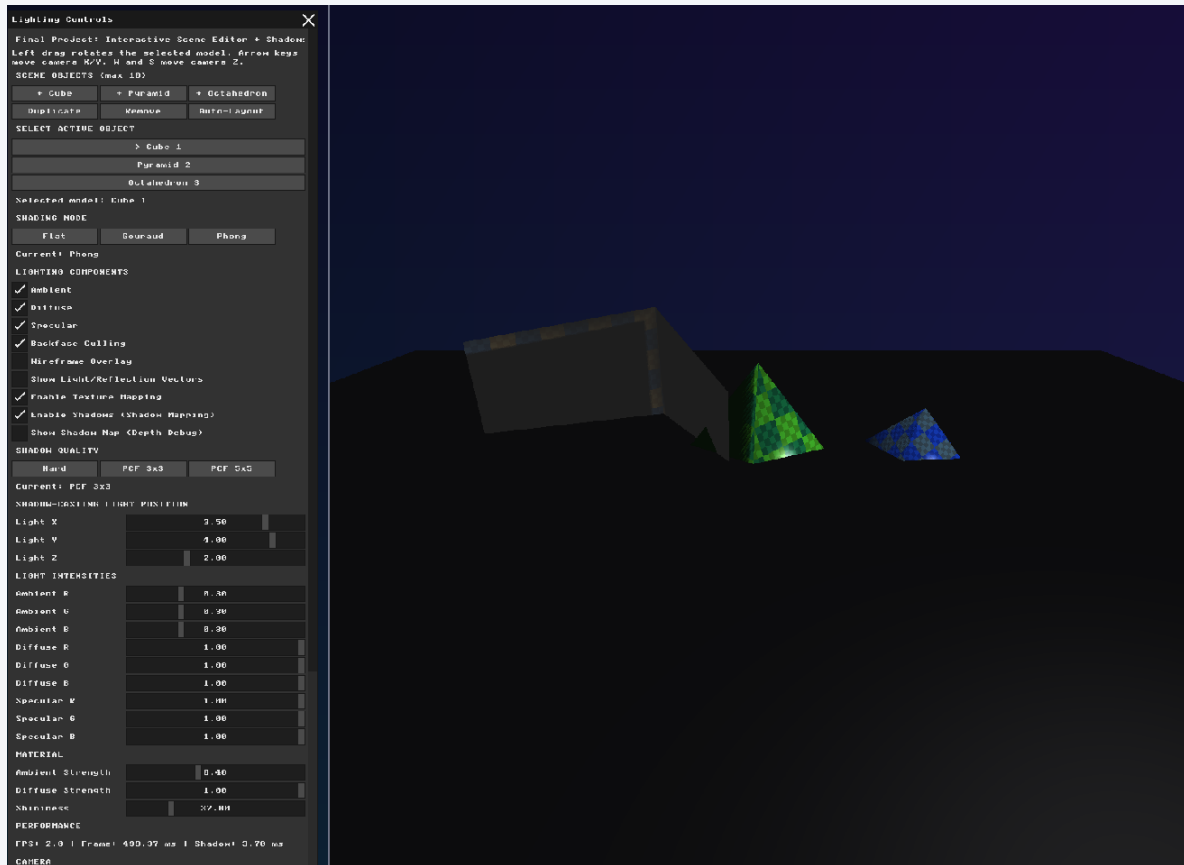


Figure 1. Final scene editor overview with multiple objects, GUI controls, ground plane, and shadows.

5. Real-Time Shadow Mapping

The central extension of the final project is real-time shadow mapping. The method uses two rendering passes. First, the scene is rendered from the light's point of view and only depth is stored. During the normal camera pass, every shaded world-space position is projected into the light's coordinate system and compared against the stored light-space depth.

Shadow pass

- A 1024 x 1024 ShadowMap depth buffer is cleared to the far value.
- A light view matrix is built by aiming the light camera toward the scene center.
- A perspective light projection matrix defines the light frustum.
- Each shadow-casting triangle is transformed into light clip space.
- The triangle is rasterized and its nearest depth is written into the shadow map.

Camera pass shadow test

```
world_position
-> light_view_projection * position
-> perspective divide
-> shadow-map UV + depth
-> compare current depth with stored depth
-> apply bias / PCF
-> shadow factor
```

Ambient lighting remains visible in shadowed regions, while diffuse and specular contributions are attenuated. This prevents shadowed objects from becoming completely black and is consistent with the lighting model used throughout the project.

Slope-dependent bias

A small depth bias is added before comparison to reduce shadow acne caused by finite depth precision and self-shadowing. The bias depends on the surface orientation relative to the light direction so that shallow angles receive a slightly larger correction.

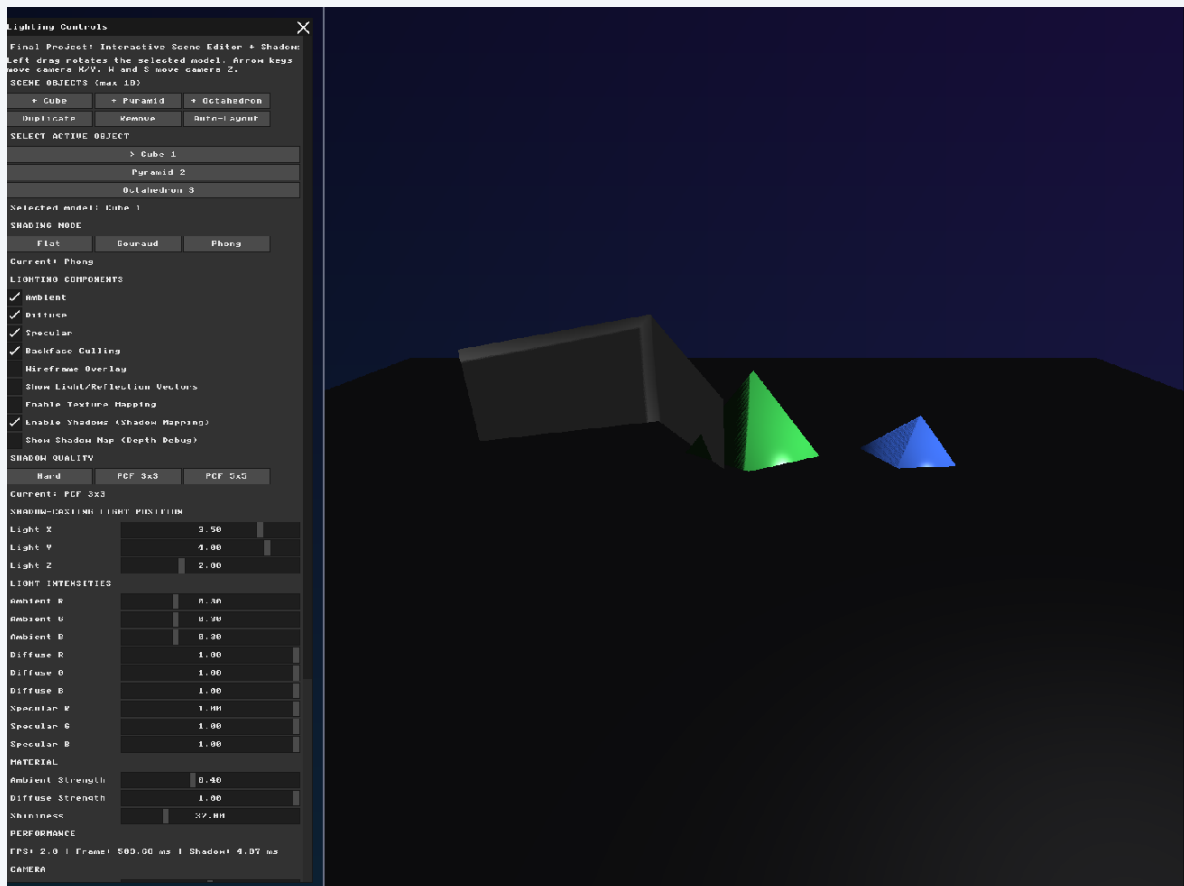


Figure 2. Real-time shadows cast by the scene objects onto the ground plane.

Ground Plane

A procedural ground quad was added below the scene to provide a stable shadow receiver. This makes it easy to observe shadow direction, filtering quality, bias behavior, and the effect of moving the light source.

6. Shadow Quality and Debug Visualization

Selectable filtering

The user can switch between three shadow sampling modes in real time. This provides a direct visual comparison between hard binary shadow edges and progressively smoother percentage-closer filtering (PCF).

Mode	Sampling	Expected Result
Hard Shadows	1 depth comparison	Sharp edges; lowest cost.
PCF 3x3	9 neighboring comparisons	Softer edges with moderate cost.
PCF 5x5	25 neighboring comparisons	Smoother edges with higher cost.

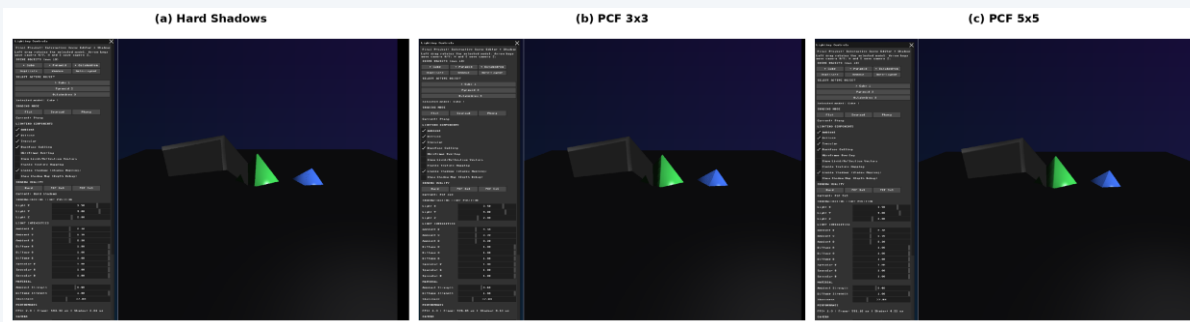


Figure 3. Comparison of Hard, PCF 3x3, and PCF 5x5 shadow quality.

Shadow-Map Depth Debug

The GUI includes a Show Shadow Map (Depth Debug) option. When enabled, the normal scene view is replaced by a grayscale visualization of the light's depth buffer. Near and far light-space depths map to different gray intensities, exposing the internal data used by the shadow test.

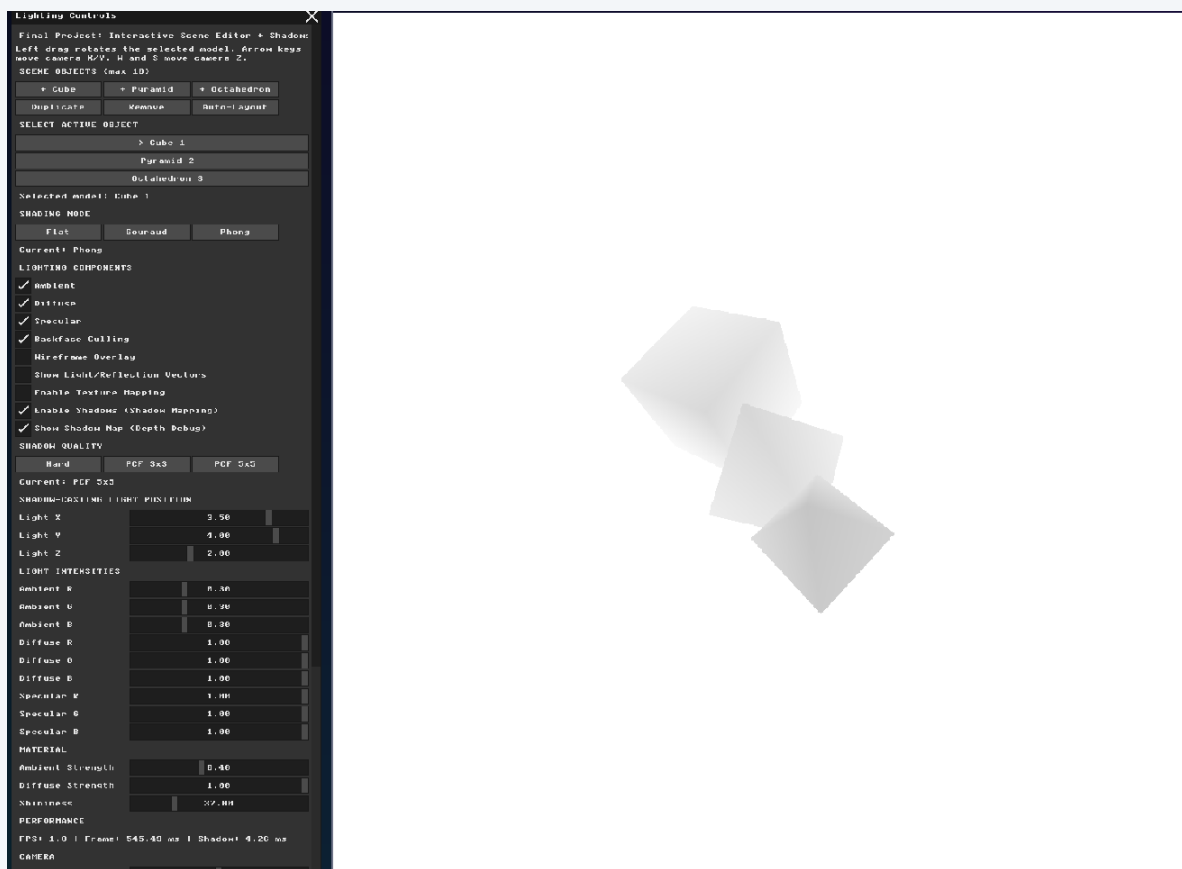


Figure 4. Shadow-map depth buffer visualized in grayscale.

7. Texture Mapping and Perspective-Correct Interpolation

Real BMP texture loading

The renderer includes `load_bmp_texture()`, which reads uncompressed 24-bit and 32-bit BMP files. The project ships with `models/checker.bmp`. If the external file cannot be loaded, the procedural checkerboard texture is retained as a fallback so the application can continue to run.

Perspective-correct attributes

Standard screen-space barycentric interpolation is not sufficient for attributes on triangles with strong perspective depth variation. Therefore, each projected vertex stores reciprocal clip-space W . The barycentric weights are corrected with $1/W$ before interpolating UV coordinates, world positions, vertex normals, and Gouraud colors.

```

q0 = alpha / w0
q1 = beta  / w1
q2 = gamma / w2
sum = q0 + q1 + q2
corrected0 = q0 / sum
corrected1 = q1 / sum
corrected2 = q2 / sum

```

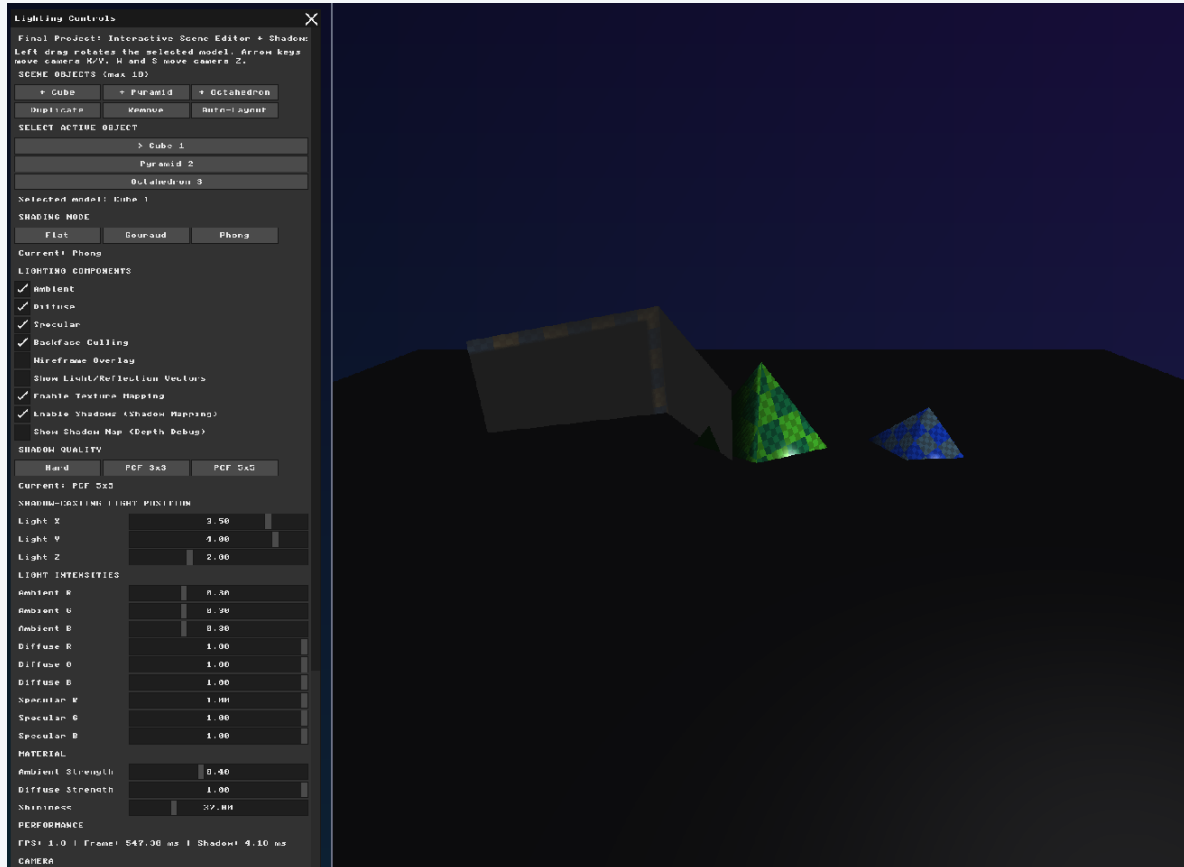


Figure 5. Textured objects rendered with perspective-correct UV interpolation.

8. Performance Measurements

The user interface reports FPS, total frame time, and shadow-pass time. These measurements are useful for demonstrating the cost of additional filtering samples and for comparing rendering with shadows enabled and disabled.

Test	FPS	Frame Time (ms)	Shadow Pass (ms)
Shadows OFF			N/A
Hard Shadows	2.0	493.93	3.01
PCF 3x3	1.9	524.25	4.13
PCF 5x5	1.0	551.18	4.22

The values above will be filled after the final successful build and run.

9. Interaction, Testing, and Final Results

Main controls

- Arrow keys move the camera in X/Y.
- W / S move the camera in Z.
- Left-drag inside the 3D viewport rotates the active model.
- Reset Camera restores the default camera.
- Reset Active Model restores the selected object's transformation.
- GUI controls edit scene objects, shading, lighting, materials, textures, shadows, and debugging views.

Final verification checklist

- ☐ Project configures and builds successfully with CMake.
- ☐ All scene objects render correctly with Z-buffer visibility.
- ☐ Object add / duplicate / remove / auto-layout operations work.
- ☐ Flat, Gouraud, and Phong shading modes can be switched in real time.
- ☐ BMP texture loads and maps correctly.
- ☐ Shadows appear on the ground plane and move with the light.
- ☐ Hard / PCF 3x3 / PCF 5x5 modes visibly differ.
- ☐ Shadow-map debug view displays a valid grayscale depth map.
- ☐ FPS and timing values update during execution.

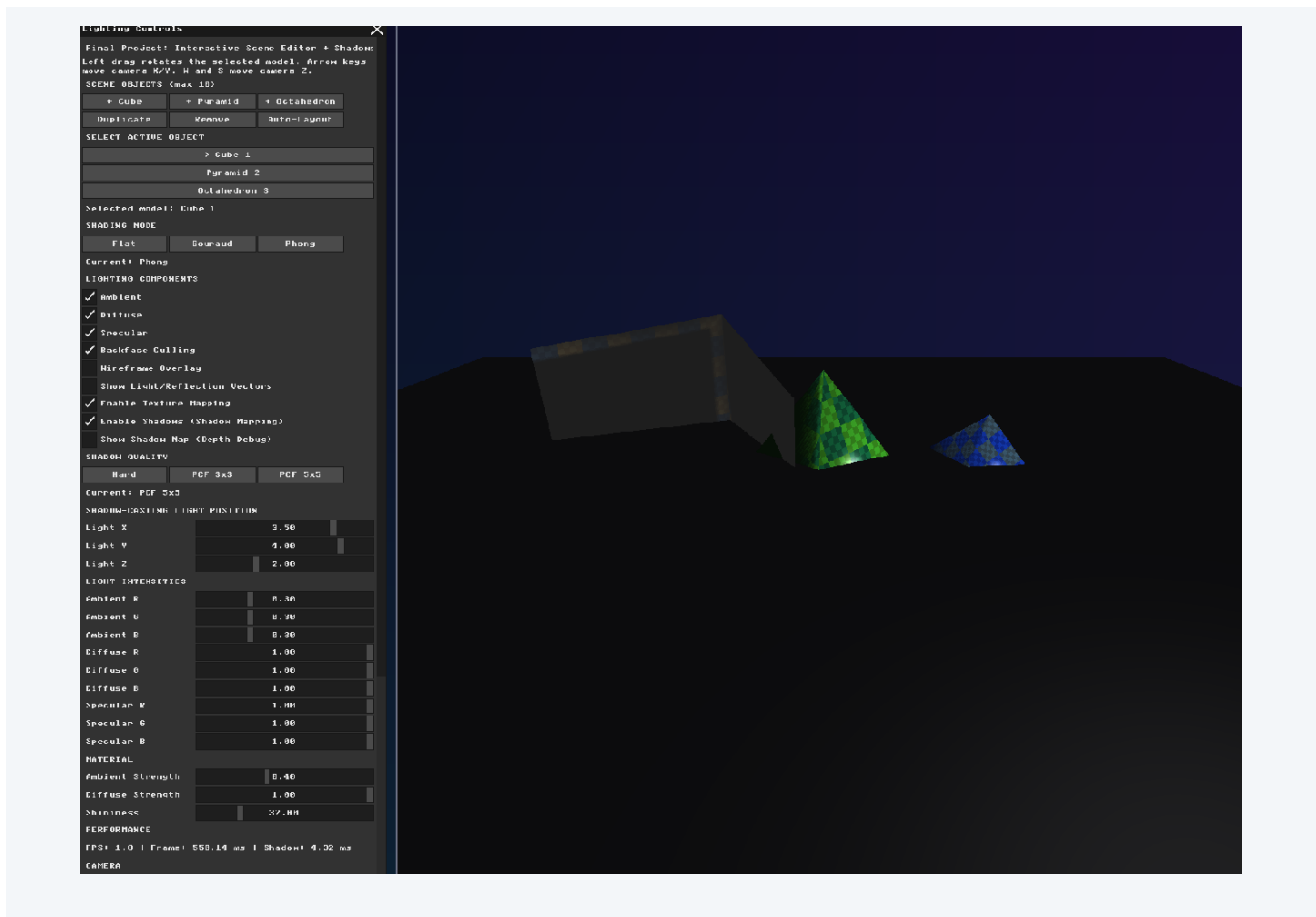


Figure 6. Final tested project window after all features are verified.

10. Known Limitation and Future Work

The current shadow system uses one perspective light camera aimed toward the scene center. Geometry outside that light frustum is treated as lit. A true omnidirectional point-light shadow system would require six light-space views arranged as a cubemap. This was intentionally left outside the current implementation in order to keep the final software renderer stable, understandable, and easy to demonstrate.

Possible future extensions

- Omnidirectional cubemap shadow mapping for point lights.
- Multiple light sources and directional / spot lights.
- Larger model library and external OBJ import from the UI.
- Additional texture filtering methods.
- Saving and loading complete scene configurations.

11. Conclusion

The final project integrates the core topics of the computer graphics course into one interactive software renderer and extends them with a complete shadow-mapping pipeline. The application supports editable 3D scene objects, model/view/projection transformations, triangle rasterization, depth buffering, multiple shading modes, materials, texture mapping, real-time shadows, debug visualization, and performance measurements. The result demonstrates how the individual algorithms developed in the homework assignments combine into a coherent rendering system without relying on a high-level graphics engine.